

IEF2I



E D U C A T I O N

Cahier des charges

Python pour tests d'intrusion

BENYAMINA Mohamed
mohamedbenyamina@formateur.ief2i.fr
06/07/2026

Sommaire

1. Contexte et objectifs pédagogiques	2
2. Cadre légal et éthique	3
3. Périmètre fonctionnel	4
3.1 Socle obligatoire (attendu par toutes les équipes)	4
3.2 Fonctionnalité bonus (à choisir une seule)	5
3.3 Hors périmètre (explicitement exclu)	5
4. Contraintes techniques	6
5. Livrables attendus	6
6. Planning indicatif sur 3 jours	7
7. Modalités d'évaluation	8
8. Modalités de rendu	8
9. Ressources et points de départ	9

1. Contexte et objectifs pédagogiques

Ce projet conclut le cours « Python pour la cybersécurité et le pentesting ». Il vous demande de mobiliser, sur un cas concret et resserré, les compétences vues en formation : programmation Python structurée, manipulation réseau, requêtes HTTP, et production d'un livrable professionnel.

L'objectif n'est pas de construire un outil d'attaque, mais un outil d'audit défensif : un mini-auditeur en ligne de commande qui scanne des cibles autorisées, analyse leur exposition, et produit un rapport exploitable — exactement le type d'outil qu'un pentester ou un analyste sécurité junior peut être amené à écrire en début de mission.

À l'issue du projet, chaque équipe doit être en mesure de démontrer un outil fonctionnel, d'expliquer ses choix techniques, et de restituer des résultats clairs à un public non technique comme à un public technique.

2. Cadre légal et éthique

Ce projet reprend les principes vus en cours d'introduction. Ils sont impératifs et conditionnent la validation du projet, indépendamment de la qualité technique du code produit.

Cibles autorisées pour ce projet (aucune autre cible n'est acceptée) :

- scanme.nmap.org (domaine mis à disposition publiquement par le projet Nmap pour l'entraînement au scan de ports) ;
- une machine virtuelle ou un service local que l'équipe héberge elle-même (ex. localhost, conteneur Docker) ;
- ou un domaine/serveur dont un membre de l'équipe est propriétaire. Toute autre cible est strictement interdite dans le cadre de cet exercice.

Aucun test ne doit être exécuté contre un système tiers sans autorisation écrite explicite de son propriétaire.

L'outil produit doit rester strictement passif/défensif : scan et analyse en lecture seule, aucune exploitation, aucune modification de la cible.

Le rapport de projet doit mentionner explicitement la ou les cibles utilisées pour les tests.

Le non-respect de ce cadre entraîne l'invalidation du projet, quelle que soit la qualité du code.

3. Périmètre fonctionnel

Le projet est volontairement calibré pour être réalisable en 3 jours par une à quatre personnes. Il se compose d'un socle obligatoire, identique pour toutes les équipes, et d'une fonctionnalité bonus à choisir pour aller plus loin.

3.1 Socle obligatoire (attendu par toutes les équipes)

- **Module 1 (Scanner de ports)** : balayage TCP multi-thread d'une cible (IP ou nom d'hôte) sur une plage de ports paramétrable, avec distinction ouvert / fermé / filtré.
- **Module 2 (Analyseur d'en-têtes HTTP)** : pour une liste d'URL fournie en entrée, récupération des en-têtes de sécurité (Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options...) et calcul d'un score simple de bonne pratique.
- **Module 3 (Générateur de rapport)** : consolidation des résultats des deux modules précédents dans un rapport lisible (Markdown ou HTML), horodaté, mentionnant la ou les cibles auditées.
- **Interface** : le tout est pilotable depuis une interface en ligne de commande (CLI) avec des arguments explicites (ex. cible, plage de ports, fichier de sortie).
- **Gestion des erreurs** : timeouts réseau, cibles injoignables ou URL invalides doivent être gérés sans faire planter le programme.

3.2 Fonctionnalité bonus (à choisir une seule)

Chaque équipe choisit une fonctionnalité supplémentaire selon son niveau d'avancement et ses affinités.

- **Vérification TLS** : contrôle de la validité et de la date d'expiration du certificat TLS d'un site.
- **Mini-crawler défensif** : à partir de BeautifulSoup, repérage de fichiers/chemins sensibles exposés par erreur (ex. `/.git`, `/.env`, `robots.txt`) sur un domaine autorisé.
- **Interface web légère** : une page Flask permettant de lancer un audit et d'afficher le rapport dans le navigateur, en plus de la CLI.
- **Rapport enrichi** : export PDF avec mise en forme et graphique de synthèse (ex. répartition des ports ouverts).
- **Vérification d'intégrité** : calcul et comparaison de hachages (SHA-256) sur un ensemble de fichiers pour détecter une modification.

3.3 Hors périmètre (explicitement exclu)

- Toute fonctionnalité d'exploitation active (reverse shell, injection, brute force de mots de passe réels, déni de service).
- Tout test contre une cible autre que celles listées en section 2.
- La persistance de données utilisateur ou la création de comptes/API tierces payantes non fournies par le formateur.

4. Contraintes techniques

- Langage : Python 3.10 ou supérieur.
- Bibliothèques autorisées : bibliothèque standard, `requests`, `beautifulsoup4`, `argparse` ; toute autre dépendance doit être justifiée dans le README.
- Gestion de versions : le projet doit être versionné avec Git (historique de commits cohérent, pas un unique commit final).
- Qualité de code : respect des conventions PEP 8, code organisé en fonctions/modules, pas de script monolithique de plusieurs centaines de lignes.
- Portabilité : le projet doit pouvoir être installé et exécuté à partir des seules instructions du README, sur une machine vierge disposant de Python.
- Sécurité du code lui-même : aucun identifiant, jeton ou mot de passe en dur dans le code source.

5. Livrables attendus

Livrable	Contenu attendu
Code source	Dépôt Git (archive .zip ou lien) contenant l'intégralité du code et son historique de commits.
README	Instructions d'installation, de configuration et d'utilisation ; liste des dépendances ; cibles utilisées pour les tests.
Rapport de projet	2 à 4 pages : choix techniques, répartition du travail, difficultés rencontrées, limites connues, respect du cadre légal.
Exemple de rapport généré	Un exemple réel de sortie produite par l'outil sur une cible autorisée.
Soutenance orale	5 à 10 minutes : démonstration live de l'outil, suivie de questions.

6. Planning indicatif sur 3 jours

Jour	Période	Objectif
Jour 1	Matin	Cadrage : lecture du cahier des charges, répartition des rôles, mise en place de l'environnement (venv, dépôt Git), squelette de la CLI.
	Après-midi	Développement et test du scanner de ports (Module 1) sur une cible autorisée.
Jour 2	Matin	Développement de l'analyseur d'en-têtes HTTP (Module 2) ; choix de la fonctionnalité bonus.
	Après-midi	Génération du rapport (Module 3) ; intégration des modules ; début de la fonctionnalité bonus.
Jour 3	Matin	Finalisation, gestion des erreurs, rédaction du README et du rapport de projet.
	Après-midi	Tests finaux, préparation et passage des soutenances.

Ce planning est une base de référence ; chaque équipe peut l'ajuster selon son organisation interne, à condition de respecter les jalons de fin de journée.

7. Modalités d'évaluation

Le projet est noté sur 20 points.

Critère	Points
Fonctionnalités obligatoires opérationnelles (scan, analyse HTTP, rapport)	8
Qualité du code (structure, lisibilité, gestion des erreurs, PEP 8)	4
Fonctionnalité bonus choisie	3
Documentation (README, rapport de projet)	3
Soutenance et démonstration	2

Toute violation du cadre légal et éthique décrit en section 2 entraîne, quel que soit le score obtenu sur les autres critères, l'invalidation du projet.

8. Modalités de rendu

- Format : archive .zip du dépôt Git, ou lien vers un dépôt distant (GitHub/GitLab) accessible au formateur.
- Nom de l'archive ou du dépôt : **NomEquipe_MiniAuditeur** (ex. **Equipe3_MiniAuditeur**).
- Contenu minimal à la racine : code source, README.md, rapport de projet (PDF ou Markdown), exemple de rapport généré.
- Échéance : avant le début des soutenances du jour 3 après-midi ; tout rendu déposé après le début des soutenances de l'équipe n'est plus modifiable.
- Chaque membre de l'équipe doit être en mesure de présenter et d'expliquer n'importe quelle partie du code lors de la soutenance.

9. Ressources et points de départ

- Modules du cours réutilisables directement : gestion des sockets (3.1), scan de ports (3.2), requêtes HTTP (5.3), bibliothèques web (5.1).
- Cible d'entraînement recommandée pour le scanner de ports : scanme.nmap.org, mise à disposition publiquement à cet effet par le projet Nmap.
- Documentation officielle à consulter : docs Python ([argparse](#), [socket](#), [concurrent.futures](#)), documentation [requests](#) et [BeautifulSoup](#), guide des en-têtes de sécurité HTTP de l'OWASP.